

Développement Mobile

Polycopié de cours

Ngomde Cadet Kamdaou

18 mai 2026

Support de cours destiné aux étudiants de Licence en Informatique

Avant-propos. Ce polycopié accompagne le cours de développement mobile. Il présente les fondements théoriques nécessaires à la compréhension de la programmation d'applications Android avec le langage Kotlin, ainsi que des exemples de code et des exercices d'application. Les corrigés ne sont pas fournis ; les solutions seront discutées en séance de travaux dirigés.

Table des matières

1	Introduction générale au développement mobile	3
1.1	L'écosystème mobile	3
1.2	Typologie des applications	3
1.3	Architecture du système Android	4
1.4	Les composants d'une application Android	4
2	Environnement de développement et langage Kotlin	7
2.1	La chaîne d'outils	7
2.2	Processus de compilation	7
2.3	Bases du langage Kotlin	8
2.3.1	Variables et null-safety	8
2.3.2	Fonctions	9
2.3.3	Classes et classes de données	9
3	Interfaces utilisateur	11
3.1	Approches impérative et déclarative	11
3.2	Layouts principaux	11
3.3	Cycle de rendu	12
3.4	Exemple Jetpack Compose	12
3.5	Material Design	12
4	Cycle de vie et navigation	14
4.1	Cycle de vie d'une Activity	14
4.2	Intents	15
4.3	Exemple de navigation	15
4.4	Pile d'activités	15
4.5	Jetpack Navigation Component	15
5	Persistance des données	17
5.1	Choix d'un mécanisme	17
5.2	Modèle relationnel	17
5.3	Room en pratique	18
5.4	Sérialisation JSON	19
6	Réseau et API REST	20
6.1	Modèle client-serveur	20
6.2	Architecture REST	20
6.3	Programmation asynchrone	21
6.4	Retrofit	22
6.5	Sécurité réseau	22
7	Capteurs et services système	24
7.1	Modèle de permissions	24
7.2	Demande de permission au moment d'exécution	24
7.3	Capteurs	25
7.4	Notifications	25
7.5	Tâches en arrière-plan	25

8	Architecture et publication	27
8.1	Patterns architecturaux	27
8.2	MVVM en détail	27
8.3	Principes SOLID	28
8.4	Injection de dépendances	28
8.5	Tests	28
8.6	Publication sur le Play Store	29

1. Introduction générale au développement mobile

1.1 L'écosystème mobile

Le marché mondial des systèmes d'exploitation mobiles est dominé par deux acteurs : **Android** (Google), qui équipe environ 70 % des smartphones, et **iOS** (Apple), qui en équipe environ 28 %. D'autres systèmes existent (HarmonyOS, KaiOS) mais restent marginaux. Les modèles économiques associés aux applications mobiles sont multiples : applications gratuites financées par la publicité, applications payantes, modèle *freemium* avec achats intégrés, ou abonnements récurrents.

Pour comprendre : pourquoi cibler en priorité Android

Pour un développeur africain ou francophone, Android présente plusieurs avantages concrets : la part de marché y dépasse souvent 85 % en Afrique subsaharienne, les appareils sont accessibles dès 50 \$, l'inscription développeur ne coûte que 25 \$ (contre 99 \$/an pour iOS), et les outils de développement (Android Studio, SDK) sont entièrement gratuits et fonctionnent sur Windows, macOS et Linux. Apprendre d'abord Android, c'est apprendre une compétence immédiatement utilisable sur le marché local.

1.2 Typologie des applications

On distingue quatre grandes catégories d'applications mobiles :

- **Applications natives** : développées spécifiquement pour une plateforme avec ses outils officiels. Pour Android, on utilise Kotlin ou Java ; pour iOS, Swift ou Objective-C. Performance optimale, accès complet aux API.
- **Applications hybrides** : technologies web (HTML, CSS, JavaScript) embarquées dans un conteneur natif (Ionic, Cordova).
- **Applications web mobiles** : sites web responsive, éventuellement sous forme de *Progressive Web Apps* (PWA).
- **Applications cross-platform** : un code unique compilé pour plusieurs plateformes. Exemples : Kotlin Multiplatform, Flutter, React Native, .NET MAUI.

Analogie : construire une maison

Choisir entre natif et cross-platform, c'est comme choisir comment construire une maison. Le **natif**, c'est faire appel à un maçon local qui connaît parfaitement le terrain, les matériaux locaux et les techniques régionales : le résultat est parfaitement adapté, mais si vous voulez la même maison ailleurs, il faut tout recommencer. Le **cross-platform**, c'est utiliser des plans standardisés et des matériaux préfabriqués : on construit plus vite et partout pareil, mais certains détails seront moins bien adaptés au terrain local.

Comparaison : performance et accès matériel

Critère	Natif	Cross-platform	Hybride/Web
Performance	Excellente	Très bonne	Moyenne
Accès capteurs	Complet	Très bon	Limité
Coût développement	Élevé (×2)	Modéré	Faible
Look natif	Parfait	Bon	Approximatif

1.3 Architecture du système Android

Android est un système d'exploitation construit en couches au-dessus d'un noyau Linux modifié :

1. **Noyau Linux** : gestion du matériel, des processus et de la mémoire.
2. **HAL** (*Hardware Abstraction Layer*) : interface entre le matériel et le logiciel.
3. **Bibliothèques natives** en C/C++ (SQLite, OpenGL, WebKit...).
4. **ART** (*Android Runtime*) : machine virtuelle exécutant le bytecode DEX.
5. **Framework Java/Kotlin** : API publiques accessibles aux développeurs.
6. **Applications** : système et tierces.

Pour comprendre : pourquoi un noyau Linux sur un téléphone

Linux est un noyau libre, mature, robuste et bien éprouvé sur les serveurs depuis 30 ans. Google n'a pas réinventé la roue : il a pris Linux et l'a adapté pour le mobile (gestion fine de l'énergie, modèle de sécurité par application). Cela explique pourquoi Android et un serveur Linux partagent des concepts : utilisateurs Unix, permissions de fichiers, processus, signaux. Chaque application Android tourne d'ailleurs sous un *utilisateur Linux différent*, ce qui assure leur isolation.

Analogie : l'immeuble à étages

Imaginez l'architecture d'Android comme un immeuble. Le **noyau Linux** est la fondation et les fondations souterraines : on n'y touche jamais, mais tout repose dessus. La **HAL**, ce sont les conduites et câbles : elles relient le bâtiment au monde extérieur (eau, électricité = matériel). Les **bibliothèques natives** sont les services techniques de l'immeuble (chauffage, ascenseur). L'**ART** est le concierge qui parle aux résidents. Le **framework** est l'aménagement des appartements (cuisine équipée, salle de bain) que chaque locataire utilise. Les **applications**, c'est vous, les locataires.

1.4 Les composants d'une application Android

Une application Android est un assemblage de quatre types de composants :

- **Activity** : un écran avec interface utilisateur.
- **Service** : une tâche exécutée en arrière-plan, sans interface.
- **BroadcastReceiver** : un récepteur d'événements système (batterie faible, SMS reçu, connectivité changée...).
- **ContentProvider** : un mécanisme de partage de données entre applications.

À retenir

Tous les composants d'une application doivent être déclarés dans le fichier `AndroidManifest.xml`, qui sert de descripteur de l'application.

Pour comprendre : pourquoi pas de fonction `main()` sur Android

En programmation classique (C, Python, Java desktop), un programme commence par une fonction `main()` qui s'exécute du début à la fin, puis le programme se termine. **Sur mobile, ce modèle ne marche pas, pour trois raisons :**

- 1. Les ressources sont limitées.** Un smartphone a peu de RAM, une batterie qui s'épuise, et tourne souvent dix à vingt applications en parallèle. Si chaque application avait son `main()` tournant en permanence, le téléphone serait saturé en quelques minutes.
- 2. L'utilisateur passe son temps à interrompre.** Un SMS arrive, un appel sonne, on bascule sur WhatsApp, on revient. . . Une application mobile doit pouvoir être mise en pause à tout moment et reprendre proprement. Un `main()` linéaire ne sait pas faire ça.
- 3. C'est le système qui dirige, pas l'application.** On parle d'*inversion de contrôle* : votre application déclare ses composants au système, et c'est le système qui décide quand les activer, les suspendre, les détruire. Vous n'écrivez plus "*je démarre maintenant et je m'arrête là*", mais "*voici mes composants, appelle-moi quand tu en as besoin*".

Analogie : le restaurant et le serveur

Pensez à un restaurant. Vous ne décidez pas quand le serveur vient prendre votre commande : c'est *lui* qui vient *vous voir* quand il est disponible. Android fonctionne pareil. Votre code ne prend jamais l'initiative ; il *réagit* quand Android l'appelle. `onCreate()`, `onResume()`, `onPause()` sont les moments où le « serveur » (le système) vient vous parler. Entre ces moments, vous attendez patiemment, et surtout vous *consommez peu d'énergie*.

Comparaison : programme desktop vs application Android

Programme desktop / Python	Application Android
Point d'entrée unique <code>main()</code>	Plusieurs points d'entrée (composants)
S'exécute du début à la fin	Le système décide du cycle de vie
Contrôle le déroulement	Réagit aux appels du système
Se termine quand l'utilisateur quitte	Peut être tué silencieusement par Android
État conservé en mémoire	État peut être perdu à tout moment

Exercices

1. Citer les six couches de l'architecture Android et préciser le rôle de chacune.
2. Quelle est la différence fondamentale entre une application native et une application cross-platform ? Donner un avantage et un inconvénient de chaque approche.
3. Pour chaque scénario suivant, indiquer quel composant Android convient :
 - (a) Afficher la liste des produits d'une boutique.
 - (b) Lire de la musique pendant que l'utilisateur navigue dans d'autres applications.
 - (c) Réagir à l'arrivée d'un SMS.
 - (d) Partager les contacts avec une autre application autorisée.

4. Rechercher la part de marché actuelle d'Android et iOS dans votre pays et comparer avec les chiffres mondiaux.

2. Environnement de développement et langage Kotlin

2.1 La chaîne d'outils

Le développement Android moderne repose sur les outils suivants :

- **Android Studio** : environnement de développement intégré officiel, basé sur IntelliJ IDEA.
- **Android SDK** : ensemble des bibliothèques, outils de compilation et documentation.
- **JDK** (*Java Development Kit*) : version 17 ou supérieure recommandée.
- **Gradle** : système de build qui orchestre la compilation, les dépendances et le packaging.
- **Émulateur AVD** ou appareil physique pour tester.

Pour comprendre : le rôle de Gradle

Quand votre projet grossit, vous avez du code Kotlin, des images, des fichiers XML, plusieurs modules, et surtout vous dépendez de bibliothèques externes (Retrofit, Room, Compose...). Gradle automatise tout cela : il télécharge les dépendances, compile le code dans le bon ordre, génère le DEX, signe l'APK. Sans Gradle, vous devriez lancer des dizaines de commandes manuellement à chaque modification. C'est l'équivalent de **Makefile** en C, **Maven** en Java ou **npm** en JavaScript.

Comparaison : IDE et outils selon le langage

Langage	IDE typique	Build
Python	PyCharm, VS Code	pip, poetry
Java	IntelliJ, Eclipse	Maven, Gradle
JavaScript	VS Code	npm, yarn
Kotlin/Android	Android Studio	Gradle

2.2 Processus de compilation

Le code Kotlin suit le chemin suivant pour devenir une application installable :

1. Fichiers **.kt** compilés en bytecode JVM (**.class**).
2. Bytecode JVM converti en bytecode DEX (**.dex**) par l'outil D8.
3. Fichiers DEX, ressources et manifeste empaquetés dans un APK ou un AAB.
4. APK signé numériquement.
5. Installation sur l'appareil via ADB ou téléchargement depuis le Play Store.

Définition : ART

Android Runtime est la machine virtuelle qui exécute les applications. Contrairement à la JVM classique, ART est optimisée pour les contraintes mobiles : mémoire limitée, autonomie de batterie, démarrage rapide.

Pour comprendre : pourquoi un format DEX et pas du **.class** Java

Le bytecode Java standard (**.class**) a été conçu pour des serveurs avec beaucoup de mémoire. Chaque classe est dans un fichier séparé, ce qui est lent à charger sur un téléphone. Le

format DEX (*Dalvik EXecutable*) regroupe toutes les classes dans un seul fichier compact, partage les constantes entre classes, et charge plus vite. Sur un smartphone avec 2 Go de RAM, ce gain est crucial.

Analogie : traduction en chaîne

Compiler du Kotlin vers une application installée, c'est comme traduire un livre français vers le swahili, mais en passant par l'anglais : Kotlin → bytecode JVM (forme intermédiaire) → DEX (forme optimisée mobile) → APK signé (livre publié) → installation. Chaque étape transforme le contenu pour qu'il soit utilisable à la destination finale.

2.3 Bases du langage Kotlin

Kotlin est un langage moderne, concis, à typage statique, totalement interopérable avec Java. Il introduit plusieurs concepts importants.

Pour comprendre : pourquoi Kotlin remplace Java sur Android

Google a officiellement adopté Kotlin comme langage préféré en 2019. Pourquoi ? Java traînait des défauts hérités des années 90 : verbeux (beaucoup de code pour peu de choses), exposé aux `NullPointerException`, syntaxe lourde pour les opérations courantes. Kotlin garde la compatibilité avec Java mais corrige ces défauts : code 2 à 3 fois plus court, null-safety à la compilation, fonctions extensions, et coroutines natives pour l'asynchrone. C'est Java en mieux, sans casser l'existant.

2.3.1 Variables et null-safety

```
// val : immuable (équivalent à final)
val nom: String = "Ngomde"
val age = 30 // type inféré

// var : mutable
var compteur: Int = 0
compteur = compteur + 1

// Null-safety
var prenom: String? = null // type nullable
val longueur = prenom?.length ?: 0
```

Le système de types distingue les références nullables (`String?`) des références non nullables (`String`). Cela élimine à la compilation la plupart des `NullPointerException`.

Pour comprendre : le bug à un milliard de dollars

Tony Hoare, l'inventeur de la référence `null` en 1965, l'a publiquement qualifiée de « bug à un milliard de dollars » en estimant les dégâts qu'elle a causés. Dans la plupart des langages (Java, C#, Python), n'importe quelle variable peut valoir `null`, et tenter d'y accéder fait planter le programme. Kotlin résout ce problème à la racine : *une variable est non-nullable par défaut*. Si vous voulez autoriser `null`, vous devez l'indiquer explicitement avec un `?`. Le compilateur vous oblige ensuite à gérer le cas `null` avant d'utiliser la valeur. Résultat : 99 % des plantages `NullPointerException` disparaissent.

Comparaison : déclaration de variable

Langage	Déclaration équivalente
Python	<code>nom = "Ngomde"</code> (dynamique, sans type)
Java	<code>final String nom = "Ngomde";</code>
JavaScript	<code>const nom = "Ngomde";</code>
Kotlin	<code>val nom = "Ngomde"</code> (type inféré)

2.3.2 Fonctions

```
fun additionner(a: Int, b: Int): Int {
    return a + b
}

// Expression unique
fun multiplier(a: Int, b: Int) = a * b

// Paramètres par défaut et arguments nommés
fun saluer(nom: String, message: String = "Bonjour") {
    println("$message, $nom !")
}

saluer(nom = "Cadet")
```

Pour comprendre : paramètres par défaut et nommés

Ces deux fonctionnalités, absentes en Java, sont précieuses. Imaginez une fonction `envoyerEmail(destinataire, sujet, corps, copie, copieCachee, priorite, signature)` avec 7 paramètres. En Java, vous devriez soit créer 20 surcharges, soit toujours passer tous les paramètres. En Kotlin, vous mettez des valeurs par défaut pour ceux qui le permettent, et l'appelant ne précise que ce qui est utile : `envoyerEmail("a@b.com", sujet = "Test", corps = "Bonjour")`. C'est lisible, sûr et compact.

2.3.3 Classes et classes de données

```
data class Etudiant(
    val matricule: String,
    val nom: String,
    var note: Double = 0.0
)

val e = Etudiant("L2-001", "Mahamat")
e.note = 15.5
println(e) // toString() auto-généré
```

Les *data classes* génèrent automatiquement `equals`, `hashCode`, `toString` et `copy`.

Analogie : la classe data, un formulaire pré-rempli

Une *data class* en Kotlin, c'est comme un formulaire administratif déjà imprimé avec tous les champs courants (nom, prénom, signature, date). En Java, vous recopiez ces champs à la main à chaque nouveau formulaire : `equals`, `hashCode`, `toString`, getters, setters... ce qui fait souvent *50 lignes pour transporter 3 données*. Kotlin imprime tout cela automatiquement.

à partir de la déclaration en *une seule ligne*.

À retenir

Kotlin est concis, sûr (null-safety) et pleinement interopérable avec Java. Il est compilé en bytecode DEX et exécuté par ART.

Exercices

1. Installer Android Studio et créer un projet "Hello World" en Kotlin. Identifier les rôles des fichiers `MainActivity.kt`, `AndroidManifest.xml` et `build.gradle`.
2. Écrire une fonction Kotlin `moyenne(notes: List<Double>): Double` qui retourne la moyenne d'une liste de notes. Gérer le cas de la liste vide.
3. Créer une `data class Livre` avec les champs `titre`, `auteur`, `annee` et `disponible`. Instancier deux livres et afficher leurs caractéristiques.
4. Expliquer la différence entre `val` et `var`, et entre `String` et `String?`.

3. Interfaces utilisateur

3.1 Approches impérative et déclarative

Deux paradigmes coexistent aujourd'hui pour décrire l'interface utilisateur Android :

- **Impérative – XML + View System** : approche historique. L'UI est décrite dans des fichiers XML, puis manipulée par du code Kotlin qui modifie ses propriétés.
- **Déclarative – Jetpack Compose** : approche moderne (2021). L'UI est une fonction de l'état applicatif ; on décrit *ce qu'il faut afficher*, le framework gère le rendu et la mise à jour.

Pour comprendre : impératif vs déclaratif

La différence est fondamentale. **Impératif** = on dit *comment* faire : « trouve le bouton, change sa couleur en rouge, change son texte, masque-le ». **Déclaratif** = on dit *quoi* montrer : « le bouton est rouge, dit "Valider", est masqué » – et le framework calcule lui-même comment passer de l'état actuel à l'état souhaité. Le code déclaratif est plus court, plus prévisible et plus facile à tester, car il décrit toujours l'état complet plutôt que des modifications successives.

Analogie : cuisiner ou commander

Impératif, c'est cuisiner soi-même : prendre la casserole, faire chauffer l'huile, ajouter l'oignon, remuer 3 minutes, ajouter la tomate... Chaque étape compte, l'ordre est crucial. **Déclaratif**, c'est commander au restaurant : « je veux un poulet rôti avec du riz », et le cuisinier se débrouille. Vous ne décrivez que le résultat voulu, pas la procédure.

Comparaison : paradigmes UI à travers les plateformes

Plateforme	Approche moderne	
Android (récent)	Jetpack Compose (déclaratif)	
iOS	SwiftUI (déclaratif)	
Web	React, Vue, Svelte (déclaratif)	Toute l'industrie converge
Desktop	Flutter, Compose Multiplatform	
multiplate- forme		
vers le déclaratif depuis 2015.		

3.2 Layouts principaux

- **LinearLayout** : enfants alignés horizontalement ou verticalement.
- **ConstraintLayout** : positionnement par contraintes relatives entre vues. Recommandé pour les UI complexes (plat, performant).
- **FrameLayout** : superposition d'éléments.
- **RecyclerView** : affichage performant de listes longues, avec recyclage des vues.

Les équivalents Compose sont respectivement **Row**, **Column**, **Box** et **LazyColumn**.

Pour comprendre : pourquoi le « recyclage » des vues

Imaginez une liste de 10 000 contacts. Si on créait une vue par contact, on consommerait toute la RAM du téléphone. **Astuce** : il n'y a qu'une douzaine de contacts visibles à l'écran à la fois. **RecyclerView** ne crée donc qu'une quinzaine de vues réelles ; quand vous faites défiler, les vues qui sortent du haut sont *recyclées* pour afficher les nouveaux éléments qui apparaissent en bas. On garde une mémoire constante, indépendante du nombre total d'éléments.

Analogie : les chaises de la classe

Vous avez 10 000 élèves inscrits mais une salle de 30 chaises. Comment faire ? Vous ne fabriquez pas 10 000 chaises. Quand un élève sort, sa chaise est libérée et le suivant s'y assoit. C'est exactement ce que fait **RecyclerView** avec les vues à l'écran.

3.3 Cycle de rendu

Chaque vue subit trois phases :

1. **Measure** : la vue détermine la taille qu'elle souhaite occuper.
2. **Layout** : la vue place ses enfants à leur position définitive.
3. **Draw** : la vue dessine son contenu sur le canvas.

Une hiérarchie de vues trop profonde dégrade les performances. **ConstraintLayout** permet de limiter la profondeur.

Pour comprendre : pourquoi 60 images par seconde

Pour qu'une animation paraisse fluide à l'œil humain, il faut au minimum 60 images par seconde, soit *16 millisecondes par image*. Si votre code de rendu prend plus de 16 ms, l'animation « saccade ». C'est pourquoi on optimise agressivement le cycle mesure-layout-draw : chaque imbrication de vue ajoute du temps, et au bout de 5 niveaux d'imbrication, vous dépassez le budget.

3.4 Exemple Jetpack Compose

```
@Composable
fun Salutation(nom: String) {
    Column(modifier = Modifier.padding(16.dp)) {
        Text(
            text = "Bonjour, $nom !",
            style = MaterialTheme.typography.headlineMedium
        )
        Spacer(modifier = Modifier.height(8.dp))
        Button(onClick = { /* action */ }) {
            Text("Cliquez ici")
        }
    }
}
```

3.5 Material Design

Material Design est le langage visuel développé par Google. Il définit :

- Une hiérarchie visuelle fondée sur l'élévation (ombres).
- Une palette de couleurs cohérente (primaire, secondaire, surface...).
- Une typographie standardisée (display, headline, body...).
- Des composants réutilisables (`Button`, `Card`, `AppBar`...).
- Des règles d'accessibilité : taille tactile minimale 48 dp, contraste suffisant.

Pour comprendre : l'unité dp et le sp

Les écrans varient énormément : un même bouton en pixels serait minuscule sur un écran haute résolution et énorme sur un vieil appareil. Le **dp** (*density-independent pixel*) résout ce problème : 48 dp font toujours *environ 9mm physiques*, quel que soit l'écran. Le **sp** (*scale-independent pixel*) est similaire mais respecte aussi la préférence « grosse police » de l'utilisateur (accessibilité). Règle d'or : **dp** pour les dimensions, **sp** pour le texte.

Exercices

1. Construire une UI Compose qui affiche un formulaire de connexion (email, mot de passe, bouton de validation).
2. Comparer la même interface réalisée en XML puis en Compose. Quel est le nombre de lignes de code dans chaque cas ?
3. Expliquer pourquoi `ConstraintLayout` est préféré à des `LinearLayout` imbriqués pour une UI complexe.
4. Citer trois composants Material Design 3 et donner leur cas d'usage typique.

4. Cycle de vie et navigation

4.1 Cycle de vie d'une Activity

Une Activity passe par plusieurs états entre sa création et sa destruction. Les méthodes de rappel (*callbacks*) du cycle de vie sont :

- `onCreate()` : création de l'Activity (initialisation, layout).
- `onStart()` : l'Activity devient visible.
- `onResume()` : l'Activity passe au premier plan, interactive.
- `onPause()` : une autre Activity prend le focus partiellement.
- `onStop()` : l'Activity n'est plus visible.
- `onDestroy()` : libération des ressources.

À retenir

L'état d'une Activity doit être sauvegardé dans `onSaveInstanceState()` afin d'être restauré dans `onCreate()` en cas de recréation (rotation d'écran, etc.).

Pour comprendre : pourquoi autant d'états

Sur desktop, un programme est « lancé » ou « fermé », c'est binaire. Sur mobile, la situation est beaucoup plus subtile : l'application peut être visible mais pas active (une boîte de dialogue est au-dessus), active mais sur le point d'être tuée (mémoire saturée), invisible mais conservée en mémoire (en arrière-plan), ou totalement détruite (mémoire libérée). Chaque transition est un événement où votre code doit faire quelque chose de précis : sauvegarder dans `onPause`, libérer la caméra dans `onStop`, restaurer dans `onCreate`... Le cycle de vie n'est pas une complication arbitraire, c'est le *prix à payer pour économiser la batterie et la mémoire*.

Analogie : une pièce de théâtre

Le cycle de vie ressemble à une représentation théâtrale. `onCreate` = vous arrivez en coulisses, vous vous préparez, vous enflez le costume. `onStart` = on tire le rideau, vous êtes visible. `onResume` = vous êtes en pleine scène, vous jouez. `onPause` = un autre acteur entre en avant-scène, vous reculez d'un pas. `onStop` = vous quittez la scène, vous êtes en coulisses. `onDestroy` = la représentation est finie, vous rangez le costume et rentrez.

Pour comprendre : la rotation d'écran, casse-tête classique

Quand l'utilisateur tourne le téléphone, par défaut Android *détruit complètement* votre Activity et la *recrée* ! Pourquoi un comportement aussi brutal ? Parce que le layout en mode portrait peut être radicalement différent du mode paysage (plus de colonnes, autre disposition). Au lieu de gérer une infinité de cas spéciaux, Android réinitialise tout. Conséquence : tous vos champs de saisie seraient vidés, vos compteurs remis à zéro... Sauf que vous sauvez l'état dans `onSaveInstanceState()` et le restaurez dans `onCreate()`. C'est pourquoi le `ViewModel` (vu plus loin) est si pratique : il survit à ces créations.

4.2 Intents

Un **Intent** est un objet qui décrit une opération à effectuer. Il sert principalement à démarrer une Activity, démarrer un Service, ou diffuser un message. On distingue :

- **Intent explicite** : précise la classe cible. Utilisé pour la navigation interne à l'application.
- **Intent implicite** : décrit une action (par exemple `ACTION_VIEW` pour visualiser une URL). Le système choisit l'application capable de la traiter.

Analogie : appeler quelqu'un vs lancer un appel d'offres

Un **Intent explicite**, c'est appeler directement votre ami Mahamat sur son numéro : vous savez exactement à qui vous parlez. Un **Intent implicite**, c'est lancer un appel d'offres public : « je cherche quelqu'un capable d'ouvrir une page web » – plusieurs applications peuvent candidater (Chrome, Firefox, votre navigateur préféré), et le système (ou l'utilisateur) choisit. C'est ce mécanisme qui rend Android extensible : votre application peut *remplacer* le navigateur par défaut simplement en déclarant qu'elle sait gérer `ACTION_VIEW`.

4.3 Exemple de navigation

```
// Intent explicite
val intent = Intent(this, DetailActivity::class.java)
intent.putExtra("user_id", 42)
startActivity(intent)

// Récupération dans DetailActivity
val userId = intent.getIntExtra("user_id", 0)

// Intent implicite : ouvrir une URL
val webIntent = Intent(
    Intent.ACTION_VIEW,
    Uri.parse("https://example.com")
)
startActivity(webIntent)
```

4.4 Pile d'activités

Chaque `startActivity` empile une nouvelle Activity sur le *back stack*. Le bouton retour dépile l'Activity courante. Des *flags* (par exemple `FLAG_ACTIVITY_CLEAR_TOP`) permettent de modifier ce comportement.

Analogie : la pile d'assiettes

Le *back stack* est exactement une pile d'assiettes. Chaque nouvelle Activity est posée *au-dessus* (push), comme une nouvelle assiette. Quand vous appuyez sur retour, vous prenez l'assiette du dessus (pop). L'Activity en dessous redevient visible. C'est pour cela qu'on parle de structure *LIFO* (Last In, First Out).

4.5 Jetpack Navigation Component

Le composant Navigation de Jetpack propose une approche unifiée :

- Graphe de navigation décrit en XML ou via une DSL Compose.
- Arguments typés (Safe Args).

- Animations de transition.
- Liens profonds (*deep linking*).

Pour comprendre : les liens profonds (deep links)

Avez-vous déjà cliqué sur un lien dans WhatsApp et vu l'application YouTube s'ouvrir directement sur la vidéo, sans passer par le navigateur ? C'est un *deep link*. Votre application peut déclarer qu'elle gère certaines URLs (par exemple `https://monapp.com/produit/42`), et Android l'ouvre directement sur la bonne page. Cela améliore considérablement l'expérience utilisateur : un clic sur un lien partagé arrive précisément au bon écran.

Exercices

1. Créer une application à deux écrans : une liste d'utilisateurs et une page de détail. Passer l'identifiant de l'utilisateur de la liste vers le détail.
2. Ajouter des journaux (`Log.d`) dans les six callbacks du cycle de vie et observer l'ordre d'appel lors d'une rotation d'écran.
3. Écrire un Intent implicite qui permet de partager un texte via les applications installées (indice : `ACTION_SEND`).
4. Quelle est la différence entre `onPause()` et `onStop()` ? Donner un scénario réel qui distingue les deux.

5. Persistance des données

5.1 Choix d'un mécanisme

Android propose plusieurs solutions de stockage, à choisir selon le besoin :

- **SharedPreferences** : paires clé/valeur pour les petits paramètres.
- **DataStore** : remplacement moderne, asynchrone, type-safe.
- **Fichiers** : stockage interne (privé) ou externe (partagé).
- **SQLite** : base de données relationnelle locale.
- **Room** : couche d'abstraction objet sur SQLite (ORM).
- **Cloud** : Firebase Firestore, ou API REST distante.

Pour comprendre : quel mécanisme pour quel besoin

Type de donnée	Solution adaptée
Préférence (thème, langue)	DataStore
Token d'authentification	DataStore chiffré (Encrypted-DataStore)
Photo, vidéo	Fichier dans le stockage
Liste de tâches, contacts	Room (SQLite)
Cache de réponses API	Room ou fichier JSON
Données partagées entre apps	ContentProvider
Données accessibles partout	Cloud (Firebase, API REST)

Analogie : rangements dans une maison

SharedPreferences/DataStore = le tiroir où vous mettez les clés et les pièces de monnaie : petits objets, vite retrouvés. **Fichiers** = la bibliothèque où vous gardez les livres et les albums photo : objets volumineux, individualisés. **SQLite/Room** = le placard à archives administratives avec des dossiers ordonnés et indexés : gros volume de données structurées, recherches rapides. **Cloud** = le coffre-fort à la banque : accessible de partout mais nécessite une connexion.

5.2 Modèle relationnel

Les bases de données relationnelles organisent les données en **tables**, chacune ayant un schéma : un ensemble de colonnes typées. Chaque ligne est identifiée par une **clé primaire**. Des **clés étrangères** expriment les relations entre tables. La **normalisation** consiste à éviter la redondance.

Définition : SQL

Structured Query Language est le langage standard d'interrogation et de manipulation des bases relationnelles. Principales instructions : **SELECT**, **INSERT**, **UPDATE**, **DELETE**, **JOIN**, **WHERE**, **GROUP BY**.

Pour comprendre : pourquoi SQLite est partout

SQLite est la base de données la plus déployée au monde : présente dans tous les smartphones (Android et iOS), tous les navigateurs (Chrome, Firefox), de nombreux logiciels (WhatsApp pour les messages, Skype). Pourquoi cette domination ? Parce qu'elle est embarquée (pas de serveur séparé à installer), légère (la bibliothèque fait moins de 1 Mo), fiable (testée à un niveau industriel) et libre. Contrairement à MySQL ou PostgreSQL qui sont des serveurs avec lesquels on communique en réseau, SQLite est juste un *fichier* sur le disque, et votre application le lit directement.

Comparaison : SQLite vs serveur SQL classique

Critère	Différence
Forme	SQLite : un fichier. MySQL : un processus serveur.
Concurrence	SQLite : un seul écrivain. MySQL : milliers.
Réseau	SQLite : aucun. MySQL : TCP/IP.
Volume typique	SQLite : Mo à Go. MySQL : To.
Usage idéal	SQLite : mobile, embarqué, dev local. MySQL : site web, application multi-utilisateurs.

5.3 Room en pratique

Room repose sur trois éléments :

- Les **Entities** : classes Kotlin annotées qui décrivent les tables.
- Les **DAOs** (*Data Access Objects*) : interfaces qui définissent les opérations SQL.
- La **Database** : classe abstraite qui agrège les DAOs.

Pour comprendre : pourquoi Room et pas SQLite direct

Avec SQLite « brut », vous écrivez des chaînes SQL et vous convertissez à la main les résultats en objets Kotlin – code répétitif, sujet aux fautes de frappe qui passent à la compilation mais plantent à l'exécution. Room (un ORM) génère ce code pour vous : il vérifie à la compilation que vos requêtes SQL sont valides (`SELECT * FROM etudiant` → erreur si la table s'appelle `etudiants`), mappe automatiquement les résultats vers vos data classes, et expose des `Flow` pour observer les changements. Vous écrivez beaucoup moins de code, plus sûr.

```
@Entity(tableName = "etudiants")
data class Etudiant(
    @PrimaryKey val matricule: String,
    val nom: String,
    val note: Double
)

@Dao
interface EtudiantDao {
    @Query("SELECT * FROM etudiants ORDER BY nom")
```

```
fun lireTous(): Flow<List<Etudiant>>

@Insert
suspend fun inserer(etudiant: Etudiant)

@Delete
suspend fun supprimer(etudiant: Etudiant)
}
```

5.4 Sérialisation JSON

Le format JSON est omniprésent pour l'échange de données. Les bibliothèques courantes sont :

- **kotlinx.serialization** : officielle JetBrains, performante.
- **Moshi** : alternative légère.
- **Gson** : ancienne, très répandue.

Analogie : le passeport du voyageur

La sérialisation, c'est transformer un objet Kotlin (qui n'existe qu'en mémoire de votre application) en une forme *transportable* : une chaîne de caractères JSON, par exemple. C'est comme délivrer un passeport à votre objet pour qu'il puisse voyager (être envoyé sur le réseau, sauvegardé dans un fichier, partagé entre applications). À l'arrivée, la *désérialisation* reconstruit l'objet depuis le passeport. Tout repose sur un format standard et neutre : JSON, XML, Protocol Buffers...

Exercices

1. Concevoir le schéma relationnel d'une application de gestion de bibliothèque (livres, emprunteurs, emprunts). Identifier clés primaires et étrangères.
2. Implémenter avec Room une Entity **Tache** (id, titre, terminee) et un DAO permettant de lister, ajouter, modifier et supprimer des tâches.
3. Sauvegarder dans **DataStore** la préférence "thème sombre activé" (booléen) et la restaurer au démarrage de l'application.
4. Sérialiser en JSON avec **kotlinx.serialization** une liste de trois objets **Etudiant** et la désérialiser.

6. Réseau et API REST

6.1 Modèle client-serveur

La communication réseau suit un modèle client-serveur : le **client** (l'application mobile) émet une **requête**, le **serveur** la traite et renvoie une **réponse**. Le protocole le plus utilisé est HTTP (ou sa version sécurisée HTTPS). Les principaux verbes HTTP sont :

- GET : lire une ressource.
- POST : créer une ressource.
- PUT : remplacer une ressource.
- PATCH : modifier partiellement.
- DELETE : supprimer.

Analogie : le restaurant à nouveau

La communication client-serveur est exactement comme commander dans un restaurant. **Vous (client)** formulez une demande précise (« je voudrais un poulet braisé »). **Le serveur** note, transmet en cuisine, ramène le plat ou vous dit « désolé, plus de poulet » (erreur 404). Les **verbes HTTP** sont les types de demandes possibles : GET (« je voudrais voir le menu »), POST (« j'ajoute une commande »), PUT (« je remplace ma commande »), DELETE (« annulez ma commande »).

6.2 Architecture REST

REST (*Representational State Transfer*) est un style architectural pour les API web. Ses principes :

- Chaque ressource est identifiée par une URL.
- Les opérations sont standardisées (verbes HTTP).
- Le serveur est sans état (*stateless*).
- Les représentations sont typiquement en JSON.

Exemple : GET /api/etudiants/42 récupère l'étudiant d'identifiant 42 ; POST /api/etudiants avec un corps JSON le crée.

Pour comprendre : pourquoi « sans état » (stateless)

Imaginez un serveur web qui se souviendrait de chaque utilisateur connecté en permanence : avec un million d'utilisateurs, il faudrait un million d'emplacements mémoire. Pire, si le serveur redémarre, tout est perdu. REST impose le contraire : *chaque requête contient toutes les informations nécessaires* (le token d'authentification, par exemple). Le serveur ne mémorise rien entre deux requêtes. Conséquence : on peut multiplier les serveurs derrière un load balancer, n'importe lequel peut traiter la requête, et tout marche. C'est ce qui rend les API REST *scalables* (capables de servir des millions d'utilisateurs).

Comparaison : codes de statut HTTP courants

Code	Signification
2xx (200, 201)	Succès
3xx (301, 302)	Redirection
4xx (400, 401, 404)	Erreur côté client (mauvaise requête, non authentifié, non trouvé)
5xx (500, 503)	Erreur côté serveur

6.3 Programmation asynchrone

Les opérations réseau ne doivent *jamais* bloquer le thread principal (UI), sous peine de bloquer l'interface. Plusieurs approches existent :

- Threads bas niveau (à éviter aujourd'hui).
- Callbacks (difficiles à composer, « callback hell »).
- Coroutines Kotlin (recommandées) : code asynchrone qui se lit comme du code séquentiel.
- Flow : flux de valeurs asynchrones.

Pour comprendre : pourquoi ne pas bloquer le thread UI

Le thread principal (UI) est responsable d'afficher l'écran et de capter les clics. Si vous le bloquez 5 secondes pour télécharger un fichier, pendant ces 5 secondes l'écran est figé, les clics ne répondent pas. Android affichera même le fameux dialogue ANR (*Application Not Responding*) : « Cette application ne répond pas, voulez-vous la fermer ? ». C'est pour cela qu'on délègue toute opération longue (réseau, lecture de fichier, calcul lourd) à un autre thread.

Analogie : le serveur multi-tâches

Le thread UI est comme un serveur de restaurant unique. Si un client commande un « rôti de 2 heures », le serveur reste bloqué en cuisine pendant 2 heures et tous les autres clients meurent de faim. Solution : le serveur prend la commande, la donne au cuisinier (autre thread), et retourne servir les autres clients. Quand le rôti est prêt, le cuisinier signale au serveur (callback / coroutine reprend). Personne n'attend.

Comparaison : callbacks vs coroutines**Callbacks (style ancien, JavaScript classique) :**

```
fetchUser(id) { user ->
  fetchPosts(user.id) { posts ->
    fetchComments(posts[0].id) { comments ->
      // imbrication profonde, difficile à lire
    }
  }
}
```

Coroutines (style moderne, Kotlin) :

```
val user = fetchUser(id)
val posts = fetchPosts(user.id)
val comments = fetchComments(posts[0].id)
```

Même comportement asynchrone, mais code linéaire et lisible.

6.4 Retrofit

Retrofit est la bibliothèque standard pour consommer une API REST. On définit l'interface du service par des annotations :

```
interface ApiEtudiants {
    @GET("etudiants")
    suspend fun lireTous(): List<Etudiant>

    @GET("etudiants/{id}")
    suspend fun lireUn(@Path("id") id: Int): Etudiant

    @POST("etudiants")
    suspend fun creer(@Body etudiant: Etudiant): Etudiant
}

val retrofit = Retrofit.Builder()
    .baseUrl("https://api.example.com/")
    .addConverterFactory(GsonConverterFactory.create())
    .build()

val api = retrofit.create(ApiEtudiants::class.java)
```

Pour comprendre : le rôle des annotations

Vous remarquez que vous ne *programmez* pas comment construire la requête HTTP. Vous *déclarez* « cette fonction = un GET sur `etudiants/{id}` » via des annotations. Retrofit lit ces annotations à la compilation et génère automatiquement le code qui construit l'URL, envoie la requête et déséréalise la réponse. Vous écrivez 5 lignes au lieu de 50.

6.5 Sécurité réseau

- **HTTPS** est obligatoire ; HTTP en clair est bloqué par défaut depuis Android 9.
- **TLS** chiffre la connexion entre client et serveur.
- Le **certificate pinning** valide l'identité du serveur en figeant son certificat dans l'application.
- Ne jamais stocker de clé ou secret en clair dans le code source.

Analogie : HTTP vs HTTPS

HTTP, c'est envoyer une lettre par carte postale : n'importe qui sur le trajet peut lire son contenu. **HTTPS**, c'est l'envoyer dans une enveloppe scellée et chiffrée : même si quelqu'un l'intercepte, il ne peut pas la lire. Aujourd'hui, envoyer un mot de passe ou un numéro bancaire en HTTP est aussi imprudent que de le crier dans la rue.

Exercices

1. Consommer l'API publique `https://jsonplaceholder.typicode.com/users` et afficher la liste des utilisateurs dans l'application.
2. Implémenter un écran de recherche : l'utilisateur tape un mot, l'application appelle une API et affiche les résultats. Gérer le cas d'erreur réseau.
3. Expliquer pourquoi l'on utilise une **coroutine** (`suspend fun`) pour appeler une API plutôt qu'un appel synchrone.

4. Comparer **GET** et **POST** : quand utiliser l'un plutôt que l'autre ?

7. Capteurs et services système

7.1 Modèle de permissions

Android catégorise les permissions selon leur sensibilité :

- **Normales** : accordées automatiquement à l'installation (Internet, vibration, etc.).
- **Dangereuses** : doivent être demandées à l'utilisateur au moment d'exécution (géolocalisation, caméra, microphone, contacts, SMS, stockage).
- **Spéciales** : nécessitent un passage par les paramètres système.

Toutes les permissions doivent être déclarées dans `AndroidManifest.xml`.

Pour comprendre : pourquoi un système de permissions

Sur un PC classique, n'importe quel programme installé peut accéder à n'importe quel fichier de l'utilisateur. Catastrophe si vous installez un programme malveillant. Sur Android, chaque application est *enfermée dans son bac à sable* (*sandbox*) et ne peut accéder qu'à ses propres données. Pour accéder aux contacts, à la caméra, à votre position, etc., elle doit *demandar explicitement* la permission. L'utilisateur peut refuser ; il garde le contrôle. Cette architecture rend Android beaucoup plus résistant aux logiciels malveillants qu'un PC classique.

Analogie : l'appartement et les clés

Imaginez un immeuble (Android) où chaque appartement (application) a sa propre serrure. Le concierge (système) garde les clés des espaces communs : la cave (stockage), le jardin (géolocalisation), l'interphone (microphone). Quand un locataire veut accéder à un espace commun, il doit demander la clé au concierge, qui transmet la demande au propriétaire (vous, l'utilisateur). C'est vous qui décidez si vous donnez accès ou non.

7.2 Demande de permission au moment d'exécution

```
val launcher = registerForActivityResult(  
    ActivityResultContracts.RequestPermission()  
) { accordee ->  
    if (accordee) {  
        // Utiliser la fonctionnalité  
    } else {  
        // Refusée : afficher un message  
    }  
}  
  
launcher.launch(Manifest.permission.ACCESS_FINE_LOCATION)
```

Pour comprendre : permission install-time vs runtime

Jusqu'à Android 5, toutes les permissions étaient demandées à l'installation : un écran listait tout, l'utilisateur acceptait en bloc ou refusait l'installation. C'était binaire et l'utilisateur ne lisait souvent rien. Depuis Android 6, les permissions dangereuses sont demandées *au moment de l'usage* : votre application peut être installée et n'a accès à la caméra qu'au moment où elle veut en prendre une photo. L'utilisateur comprend mieux le contexte et peut refuser sélectivement.

7.3 Capteurs

Les principaux capteurs disponibles sur un smartphone moderne sont :

- **Accéléromètre** : détection des mouvements et orientation.
- **Gyroscope** : vitesse de rotation.
- **GPS** : géolocalisation par trilatération satellitaire. Le récepteur calcule sa position en mesurant le délai des signaux reçus d'au moins quatre satellites.
- **Magnétomètre** : boussole.
- **Capteurs de luminosité, proximité** : adaptation au contexte.

Pour comprendre : comment le GPS calcule votre position

Chaque satellite GPS envoie en continu un signal contenant son identité et l'heure exacte d'émission. Votre téléphone reçoit ce signal après un délai correspondant à la distance satellite-téléphone. Avec un seul satellite, vous êtes sur une sphère autour de lui ; avec deux, sur un cercle (intersection de deux sphères) ; avec trois, sur deux points (l'un dans l'espace) ; avec **quatre satellites**, on détermine une position unique dans l'espace 3D. C'est la *trilatération*. Précision typique en extérieur dégagé : 3 à 5 mètres.

Analogie : la trilatération en ville

Imaginez que vous êtes perdu en ville et que vous appelez trois amis. Le premier vous dit « je t'entends à 500 m » : vous êtes sur un cercle de 500 m autour de lui. Le deuxième dit la même chose à 300 m : vous êtes à l'intersection des deux cercles, donc sur deux points possibles. Le troisième dit « 200 m » : un seul point reste. Vous savez où vous êtes. Le GPS fait exactement pareil avec les satellites, mais à la vitesse de la lumière.

7.4 Notifications

Depuis Android 8 (Oreo), chaque notification doit appartenir à un **canal** (*channel*). Les notifications peuvent porter des actions (boutons), des médias, et peuvent être déclenchées localement ou à distance via Firebase Cloud Messaging (FCM).

Pour comprendre : pourquoi des canaux

Avant Android 8, l'utilisateur ne pouvait que désactiver *toutes* les notifications d'une application – frustrant si on aime les messages WhatsApp mais pas les notifications de groupes spam. Les canaux permettent à l'utilisateur de choisir finement : il peut accepter les « messages directs » et bloquer les « messages de groupe », par exemple. Le développeur déclare les canaux, l'utilisateur choisit ce qu'il accepte.

7.5 Tâches en arrière-plan

- **Coroutines + ViewModel** : tâches liées au cycle de vie de l'UI.
- **WorkManager** : tâches différées ou périodiques, garanties d'exécution.
- **Foreground Service** : tâches longues visibles à l'utilisateur (musique, fitness, navigation).

Depuis Android 8, les services en arrière-plan classique sont fortement restreints pour préserver la batterie.

Pour comprendre : pourquoi Android tue les services en arrière-plan

Une étude de Google a montré qu'une partie importante de la consommation de batterie provenait d'applications qui restaient actives en arrière-plan sans raison (vérification de mises à jour toutes les 30 secondes, géolocalisation continue, etc.). Depuis Android 8, Android *tue agressivement* ces services fantômes. Si vous avez besoin d'une tâche périodique, vous devez passer par **WorkManager**, qui regroupe intelligemment les tâches de plusieurs applications pour réveiller le téléphone moins souvent : meilleure autonomie pour l'utilisateur, contrainte pour le développeur.

Analogie : le facteur intelligent

Avant Android 8, c'était comme si chaque application avait son propre facteur qui sonnait à votre porte 100 fois par jour. Épuisant. Depuis Android 8, il n'y a qu'un seul facteur (**WorkManager**) qui passe quelques fois par jour avec *tout le courrier groupé*. Le résultat est le même (vos applications fonctionnent), mais le téléphone et la batterie sont préservés.

Exercices

1. Demander la permission de géolocalisation, l'obtenir si accordée, et afficher la latitude et la longitude de l'appareil.
2. Implémenter une notification locale qui s'affiche lorsque l'utilisateur clique sur un bouton. Y associer une action (par exemple "Ouvrir").
3. Lire les valeurs de l'accéléromètre et afficher dynamiquement les trois axes (x, y, z).
4. Programmer avec WorkManager une tâche périodique (toutes les heures, par exemple) qui écrit dans les journaux.

8. Architecture et publication

8.1 Patterns architecturaux

Plusieurs patterns permettent de structurer une application :

- **MVC** (*Model-View-Controller*) : pattern classique.
- **MVP** (*Model-View-Presenter*) : améliore la testabilité.
- **MVVM** (*Model-View-ViewModel*) : recommandé par Google pour Android.
- **Clean Architecture** : couches indépendantes (domain, data, presentation).

Pour comprendre : pourquoi un pattern d'architecture

Sur un petit projet de 100 lignes, on peut tout mettre dans une seule classe : ça marche. Mais dès qu'on dépasse 1000 lignes, le code devient ingérable : modifier l'affichage casse la base de données, ajouter une fonctionnalité brise trois autres, personne ne sait par où commencer pour corriger un bug. Un *pattern d'architecture* est une recette éprouvée pour organiser le code en parties spécialisées qui se parlent peu. On peut alors modifier une partie sans casser les autres, tester chaque partie isolément, et plusieurs développeurs peuvent travailler en parallèle sans se marcher sur les pieds.

Analogie : la cuisine d'un restaurant

Une application sans architecture, c'est un cuisinier seul qui fait tout dans une seule grande casserole : entrée, plat, dessert. Vite débordé, vite plein. Avec **MVVM**, on a une vraie brigade : le **Model** (les fournisseurs et le stock), le **ViewModel** (le chef qui orchestre les plats), la **View** (le serveur qui présente au client). Chacun fait son métier, sans interférer avec les autres. Ajouter un nouveau plat au menu ne perturbe pas le service en cours.

8.2 MVVM en détail

- **Model** : représente les données et la logique métier.
- **View** : l'Activity, le Fragment ou le Composable. Affiche l'état et transmet les événements utilisateur.
- **ViewModel** : expose l'état observable à la View ; survit aux changements de configuration (rotation...).

Les outils Jetpack associés sont **ViewModel**, **LiveData**, **StateFlow**, et le pattern **Repository**.

Pour comprendre : le miracle du ViewModel

Vous vous souvenez du casse-tête de la rotation d'écran (Chapitre 4) ? L'Activity est détruite et recrée, et vous perdez l'état. Le **ViewModel** contourne ce problème : il vit *en dehors* du cycle de vie de l'Activity. Au moment où l'Activity est détruite par la rotation, le **ViewModel** reste en mémoire. Quand l'Activity est recrée, elle retrouve le même **ViewModel** intact, avec toutes les données déjà chargées. C'est pour cela qu'on y met l'état UI et les appels réseau : ils survivent aux rotations sans effort.

8.3 Principes SOLID

- **Single Responsibility** : chaque classe a une seule raison de changer.
- **Open/Closed** : ouvert à l'extension, fermé à la modification.
- **Liskov Substitution** : un sous-type doit pouvoir remplacer son type parent sans casser le programme.
- **Interface Segregation** : préférer plusieurs interfaces spécifiques à une grosse interface généraliste.
- **Dependency Inversion** : dépendre des abstractions, pas des implémentations.

Analogie : SOLID au quotidien

S : un couteau coupe, une fourchette pique. Pas un outil multi-fonction mal fait. **O** : pour ajouter un goût à votre cuisine, vous ajoutez une épice, vous ne refaites pas la recette. **L** : si la recette demande un « légume vert », vous pouvez utiliser des épinards ou du chou, ça doit marcher. **I** : un menu enfant ne doit pas lister les vins. **D** : la recette dit « prendre une casserole », pas « prendre la casserole rouge Tefal 24 cm que j'ai achetée en 2019 ».

8.4 Injection de dépendances

Au lieu de créer ses dépendances, une classe les reçoit de l'extérieur. Cela favorise le couplage faible, la testabilité (substitution par des mocks) et la réutilisation. Les frameworks courants sur Android sont **Hilt** (recommandé), **Koin** (simple), et **Dagger 2** (puissant mais complexe).

Pour comprendre : ce que change l'injection de dépendances

Sans DI : `class Service { val db = MaBaseDeDonnees() }`. Le service crée lui-même sa dépendance. Pour le tester, impossible de remplacer la vraie base par une fausse. Avec DI : `class Service(val db : Database)`. Le service reçoit sa dépendance de l'extérieur. En test, on lui passe une fausse base ; en production, la vraie. Code identique, comportement testable.

Analogie : le bricoleur et la boîte à outils

Sans DI, c'est un bricoleur qui fabrique lui-même chaque outil dont il a besoin avant de commencer : il forge le marteau, fond les vis, taille le tournevis... Inefficace et peu polyvalent. Avec DI, on lui présente la boîte à outils complète au début du chantier : il prend ce dont il a besoin, fait son travail. Demain, sur un autre chantier, on lui donnera une autre boîte (avec des outils plus performants peut-être). Le bricoleur reste le même.

8.5 Tests

- **Tests unitaires** : isolés, rapides, exécutés sur la JVM. Outils : JUnit, MockK.
- **Tests d'intégration** : vérifient l'interaction de plusieurs composants.
- **Tests instrumentés** : exécutés sur émulateur ou appareil. Outils : Espresso, Compose Testing.
- **TDD** (*Test-Driven Development*) : écrire le test avant le code.

Pour comprendre : pourquoi écrire des tests

Sur le moment, écrire des tests semble doubler le travail : pourquoi écrire du code qui vérifie d'autre code ? La raison apparaît au bout de quelques mois. Quand vous modifiez une fonction, comment savoir si vous n'avez pas cassé autre chose ? Sans tests, vous devez tester à la main toute l'application. Avec tests, vous lancez la suite (1 seconde) et vous voyez tout de suite ce qui plante. Les tests sont un *filet de sécurité* qui permet d'avancer vite sans peur. C'est particulièrement vrai en équipe : votre collègue modifie son code, vos tests détectent immédiatement si cela impacte le vôtre.

8.6 Publication sur le Play Store

Les étapes principales sont :

1. Générer une **clé de signature** (keystore) avec `keytool`.
2. Construire un **Android App Bundle** (.aab).
3. Créer un compte développeur Google Play (frais uniques de 25 \$).
4. Préparer la fiche Play Store : icône, captures d'écran, description, classification d'âge.
5. Soumettre et attendre la revue (typiquement quelques jours).

Définition : Signature numérique

La signature d'un APK utilise la cryptographie asymétrique : la clé privée (gardée secrète par le développeur) signe l'application ; la clé publique permet de vérifier son intégrité et son authenticité.

Pour comprendre : pourquoi signer une application

Sans signature, n'importe qui pourrait publier une fausse mise à jour de WhatsApp qui volerait vos messages. La signature résout ce problème : seul le détenteur de la clé privée originale peut publier les mises à jour, et Android vérifie cette signature à chaque mise à jour. C'est pourquoi **si vous perdez votre clé privée, vous perdez votre application** : vous ne pouvez plus publier de mise à jour, et il faudrait republier sous un autre nom. Sauvegardez *toujours* votre keystore dans un endroit sûr.

Analogie : la signature manuscrite

Une signature numérique fonctionne comme une signature manuscrite, mais en mille fois plus fiable. Quand vous signez un chèque, votre banque compare la signature à l'échantillon qu'elle a en archive : si ça correspond, le chèque est valide. Avec une signature numérique, Android compare cryptographiquement l'APK signé à la clé publique enregistrée : si ça correspond, l'application est authentique. La différence est qu'on ne peut pas *imiter* une signature numérique (impossible mathématiquement sans la clé privée), alors qu'on peut imiter une signature manuscrite.

Comparaison : APK vs AAB (Android App Bundle)

Aspect	Différence
APK	Une seule version installable, tous les écrans et toutes les langues.
AAB	Format que Google Play découpe et adapte à chaque appareil (résolution, langue, architecture CPU).
Taille téléchargée	APK : tout. AAB : seulement le nécessaire (souvent 30 à 50 % moins).
Statut	APK : pour distribution hors Play Store. AAB : obligatoire sur Play Store depuis 2021.

Exercices

1. Refactoriser une application existante pour appliquer le pattern MVVM. Mettre en évidence ce qui se trouve dans le Model, la View et le ViewModel.
2. Identifier dans une application donnée un cas de violation du principe *Single Responsibility* et proposer un découpage.
3. Écrire un test unitaire JUnit pour une fonction `calculerTVA(prixHT: Double, taux: Double): Double`.
4. Générer un keystore et signer une version release d'une application. Quelles précautions prendre concernant la clé privée ?
5. Comparer Hilt et Koin : avantages et inconvénients de chaque approche.

Fin du polycopié.